Nagar Yuwak Shikshan Sanstha's
Yeshwantrao Chavan College of Engineering
(An Autonomous Institution affiliated to Rashtrasant Tukadoji Maharaj Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu
Department of Computer Technology

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session 2025-2026

Vision: Dream of where you want.	**Mission:** Means to achieve Vision

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CL abbreviation pronounce as Pep-si-lL easy to recall**
PEO2	**Core Competence**	**E: Environment (Learning Environment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C: Core Competence**	
PEO5	**Learning Environment**	**L: Breadth (Learning in diverse areas)**	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:
Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Name and Signature of Student and Date
(Signature and Date in Handwritten)

Session	**2025-26 (ODD)**	**Course Name**	**HPC Lab**
Semester	**7**	**Course Code**	**22ADS706**
Roll No	**03**	**Name of Student**	**Debasrita Chattopadhyay**

Practical Number	**05**
Course Outcome	1. Understand and Apply Parallel Programming Concepts 2. Analyze and Improve Program Performance. 3. Demonstrate Practical Skills in HPC Tools and Environments.
Aim	**Basics of MPI Programming**
Problem Definition	MPI program demonstrates how to initialize the MPI environment, send and receive messages, and finalize the MPI environment.
Theory (100 words)	MPI (Message Passing Interface) is a standard library for parallel programming in distributed memory systems. Multiple processes (running the same or different machines) communicate through message passing. MPI fits in many of the high-performance computing (HPC) clusters that need large-scale parallelism. Note: - Processes will communicate explicitly using messages. - There is point-to-point as well as collectives.
Procedure and Execution (100 Words)	Algorithm: **1. Download OpenMPI** wget https://download.open-mpi.org/release/open-mpi/v5.0/openmpi-5.0.5.tar.gz **2.Extract the Source Code** tar -xvzf openmpi-5.0.5.tar.gz cd openmpi-5.0.5 **3. Configure for Local Installation** Tell OpenMPI to install inside your home folder: ./configure --prefix=$HOME/.openmpi **4. Compile OpenMPI**

	This will take a few minutes depending on your machine: make -j4 **5. Install Locally** make install **6. Update Your Environment** Add the following lines to the **end of your ~/.bashrc file**: export PATH=$HOME/.openmpi/bin:$PATH export LD_LIBRARY_PATH=$HOME/.openmpi/lib:$LD_LIBRARY_PATH **7. Then reload:** source ~/.bashrc **8. Verify Installation** mpicc --version mpirun --version
	Code: MPI Programs Contains 4 MPI examples: 1. Hello World 2. Broadcast 3. Scatter & Gather 4. Reduce Write a Basic MPI Program (Hello World) #include <stdio.h> #include <mpi.h> int main(int argc, char* argv[]) { int rank, size; MPI_Init(&argc, &argv); // Initialize MPI

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Get
process rank
    MPI_Comm_size(MPI_COMM_WORLD, &size); // Get total
processes

    printf("Hello from process %d of %d\n", rank, size);

    MPI_Finalize(); // Finalize MPI
    return 0;
}
```

Point-to-Point Communication (Send/Recv)

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char* argv[]) {
    int rank, size, data;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        data = 100;
        MPI_Send(&data, 1, MPI_INT, 1, 0,
MPI_COMM_WORLD);
        printf("Process 0 sent data %d to process 1\n", data);
    } else if (rank == 1) {
        MPI_Recv(&data, 1, MPI_INT, 0, 0,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process 1 received data %d from process 0\n", data);
    }

    MPI_Finalize();
    return 0;
}
```

Collective Communication (Broadcast)

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char* argv[]) {
    int rank, size, data;

    MPI_Init(&argc, &argv);
```

	`` MPI_Comm_rank(MPI_COMM_WORLD, &rank); MPI_Comm_size(MPI_COMM_WORLD, &size); if (rank == 0) { data = 42; // Root sets data } MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD); printf("Process %d received data %d\n", rank, data); MPI_Finalize(); return 0; } ```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        data = 42; // Root sets data
    }

    MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);
    printf("Process %d received data %d\n", rank, data);

    MPI_Finalize();
    return 0;
}
```

Reduce Example (Summing Values)

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char* argv[]) {
    int rank, size, local_data, total_data;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    local_data = rank + 1; // Each process has some local data

    MPI_Reduce(&local_data, &total_data, 1, MPI_INT,
MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Total sum of all data is: %d\n", total_data);
    }

    MPI_Finalize();
    return 0;
}
```

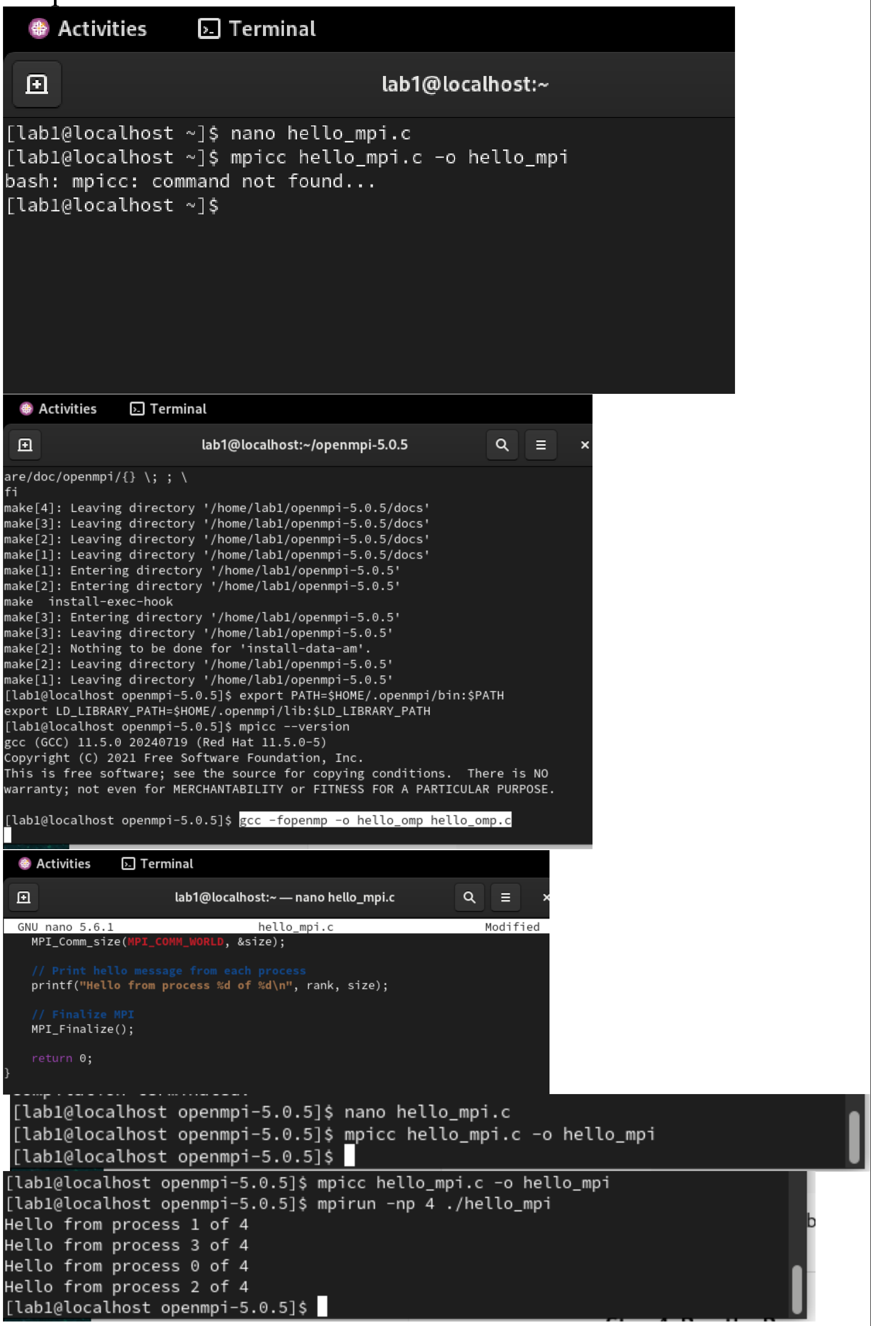
Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Output:



```
Activities   Terminal
lab1@localhost:~/openmpi-5.0.5 — nano send_recv.c
GNU nano 5.6.1                 send_recv.c                 Modified
#include <stdio.h>
#include <mpi.h>

int main(int argc, char* argv[]) {
    int rank, size, data;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        data = 100;
        MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        printf("Process 0 sent data %d to process 1\n", data);
    }
    else if (rank == 1) {
        MPI_Recv(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process 1 received data %d from process 0\n", data);
    }

^G Help    ^O Write Out  ^W Where Is  ^K Cut    ^T Execute  ^C Location
^X Exit    ^R Read File  ^\ Replace   ^U Paste  ^J Justify  ^_ Go To Line
```

```
[lab1@localhost openmpi-5.0.5]$ nano send_recv.c
[lab1@localhost openmpi-5.0.5]$ mpicc send_recv.c -o send_recv
mpirun -np 2 ./send_recv
```

```
Process 0 sent data 100 to process 1
Process 1 received data 100 from process 0
[lab1@localhost openmpi-5.0.5]$ nano broadcast.c
```

```
Activities   Terminal
lab1@localhost:~/openmpi-5.0.5 — nano broadcast.c
GNU nano 5.6.1                 broadcast.c                 Modified
#include &lt;stdio.h&gt;

#include &lt;mpi.h&gt;
int main(int argc, char* argv[]) {
int rank, size, data;
MPI_Init(&amp;argc, &amp;argv);
MPI_Comm_rank(MPI_COMM_WORLD, &amp;rank);
MPI_Comm_size(MPI_COMM_WORLD, &amp;size);
if (rank == 0) {
data = 42; // Root process sends data
}
// Broadcast data from process 0 to all other processes
MPI_Bcast(&amp;data, 1, MPI_INT, 0, MPI_COMM_WORLD);
printf(&quot;Process %d received data %d\n&quot;, rank, data);
MPI_Finalize();
return 0;
}

^G Help    ^O Write Out  ^W Where Is  ^K Cut    ^T Execute  ^C Location
^X Exit    ^R Read File  ^\ Replace   ^U Paste  ^J Justify  ^_ Go To Line
```

```
[lab1@localhost openmpi-5.0.5]$ nano broadcast.c
[lab1@localhost openmpi-5.0.5]$ mpicc broadcast.c -o broadcast
mpirun -np 4 ./broadcast
```

```
Activities    Terminal
lab1@localhost:~/openmpi-5.0.5 — nano broadcast.c
GNU nano 5.6.1                  broadcast.c                  Modified
#include <stdio.h>
#include <mpi.h>

int main(int argc, char* argv[]) {
    int rank, size, data;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        data = 42; // root process sets the data
        printf("Process 0 is broadcasting data %d\n", data);
    }

    // Broadcast data from process 0 to all processes
    MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);

    printf("Process %d received data %d\n", rank, data);

^G Help      ^O Write Out ^W Where Is  ^K Cut    ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste  ^J Justify  ^_ Go To Line
```

```
[lab1@localhost openmpi-5.0.5]$ nano broadcast.c
[lab1@localhost openmpi-5.0.5]$ mpicc broadcast.c -o broadcast
[lab1@localhost openmpi-5.0.5]$ mpirun -np 4 ./broadcast
Process 0 is broadcasting data 42
Process 0 received data 42
Process 2 received data 42
Process 1 received data 42
Process 3 received data 42
[lab1@localhost openmpi-5.0.5]$
```

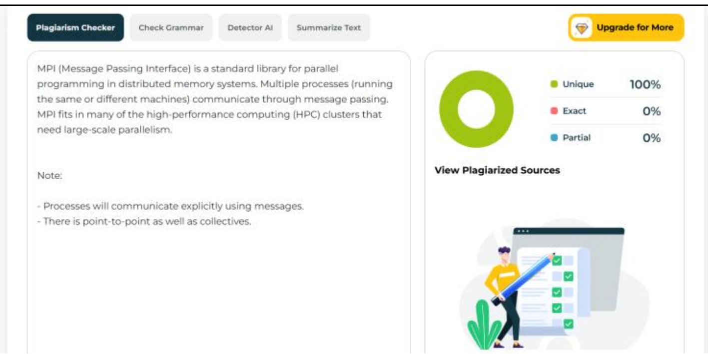
```
[lab1@localhost openmpi-5.0.5]$ nano reduce.c
```

```
Activities    Terminal
lab1@localhost:~/openmpi-5.0.5 — nano reduce.c
GNU nano 5.6.1                  reduce.c                  Modified
#include <stdio.h>
#include <mpi.h>

int main(int argc, char* argv[]) {
    int rank, size;
    int value, sum;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Each process sets a value equal to its rank
    value = rank;

    // Reduce operation: sum of all values, result stored in root (process 0)
    MPI_Reduce(&value, &sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Sum of all ranks = %d\n", sum);
    }
^G Help      ^O Write Out ^W Where Is  ^K Cut    ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste  ^J Justify  ^_ Go To Line
```

```
[lab1@localhost openmpi-5.0.5]$ nano reduce.c
[lab1@localhost openmpi-5.0.5]$ mpicc reduce.c -o reduce
```

```
Process 3 received data 42
[lab1@localhost openmpi-5.0.5]$ nano reduce.c
[lab1@localhost openmpi-5.0.5]$ mpicc reduce.c -o reduce
[lab1@localhost openmpi-5.0.5]$ mpirun -np 4 ./reduce
```

```
[lab1@localhost openmpi-5.0.5]$ mpicc reduce.c -o
[lab1@localhost openmpi-5.0.5]$ mpirun -np 4 ./red
Sum of all ranks = 6
[lab1@localhost openmpi-5.0.5]$
```

Output Analysis	Together, these programs cover basic MPI features: • Process identification (Hello World) • Point-to-point communication (Send/Recv) • Collective communication (Broadcast, Reduce)
Link of student Github profile where lab assignment has been uploaded	https://github.com/srita2003/HPC_Practicals/blob/main/03_Practical%20No_05_HPC.pdf
Conclusion	MPI Programming implemented successfully.
Plag Report (Similarity index < 12%)	
Date	**2 / 9 /25**